Universidad de los Andes

Departamento de Ingeniería de Sistemas y Computación

Laboratorio 3: Análisis capa de transporte y sockets

ISIS 2311 – REDES Y SERVICIOS DE COMUNICACIONES

Prof. Yezid Donoso

Prof. Nathalia Quiroga

Grupo 1

Juan Diego Quintero 202412750

María Inés Velásquez Acosta 202224325

Nicolás Salazar Sánchez 202124147

2026-10

Contenido

5.1. Configuración inicial 3

Todos los programas deben implementarse en lenguaje C. Se recomienda compilar con gcc y ejecutar las pruebas en un entorno Linux. 3

Creación de un directorio del laboratorio. 3

Dentro del directorio, se organizan los archivos en: 3

5.2. Pruebas y comparacion 5

Ejecute simultáneamente al menos un broker, dos suscriptores y dos publicadores. 5

Captura de tráfico con Wireshark 8

Analice las siguientes preguntas: 12

6.0. Trabajo propuesto 14

Bibliografía 20

5.1. Configuración inicial

Todos los programas deben implementarse en lenguaje C. Se recomienda compilar con gcc y ejecutar las pruebas en un entorno Linux.

Nota: Como se realizó el bono de realizar los programas en assembly, se crearon los programas tanto en C como en assembly. Ambas implementaciones se ejecutaron en máquinas virtuales Ubuntu Linux.:

TCP

Se implementaron seis archivos: tres en C (broker_tcp.c, publisher_tcp.c, subscriber_tcp.c) y tres en assembly x86-64 (broker_tcp.asm, publisher_tcp.asm, subscriber_tcp.asm). Los archivos C se compilan con gcc y los de assembly con nasm y ld directamente en la VM.

UDP

Se implementaron seis archivos: tres en C (broker_udp.c, publisher_udp.c, subscriber_udp.c) y tres en assembly (broker_udp.s, publisher_udp.s, subscriber_udp.s). Se compilan mediante Makefile con el comando make. Los archivos de captura de tráfico se encuentran en la carpeta udp_pcap_captures.

Creación de un directorio del laboratorio.

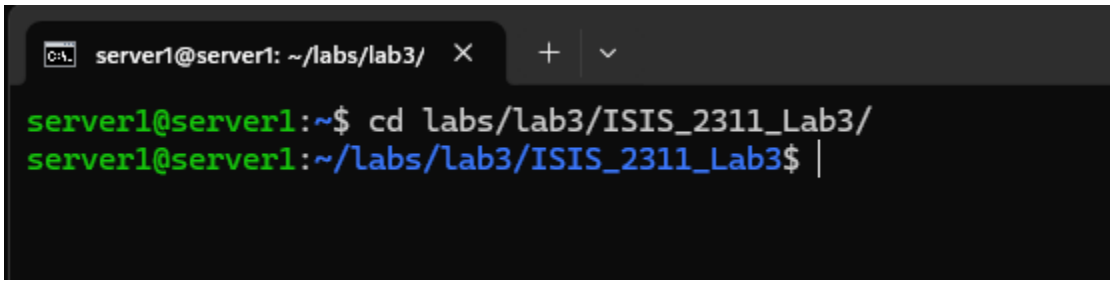

Ilustración 1 Directorio principal del laboratorio 3

Se muestra el directorio del laboratorio tras ejecutar “cd labs/lab3/ISIS_2311_Lab3/”, confirmando que la estructura de archivos está correctamente organizada en la máquina virtual.

Dentro del directorio, se organizan los archivos en:

a. broker_tcp.c , publisher_tcp.c, subscriber_tcp.c
b. brocker_udp.c, publisher_udp.c, susbscriber_udp.c

Por facilidad, se separaron los directorios en TCP y UDP, para separar los comandos de compilación, la ejecución, y por simplicidad de distinción de los protocolos.

UDP: note que todos los archivos de udp_pcap_captures corresponden a capturas de tráfico.

Considerando esto a continuación se ve la información relacionada con UDP:

```
server1@server1:~/labs/lab3/ISIS_2311_Lab3$ ls -R
.:
'Laboratorio 3 - Entregable - Copy.pdf'   README.md   udp_asm   udp_C   udp_pcap_captures

./udp_asm:
broker_udp_asm  gen_asm.sh  publisher_udp_asm  README.md           subscriber_udp.s
broker_udp.s    Makefile    publisher_udp.s    subscriber_udp_asm

./udp_C:
broker_udp.c  Makefile  publisher_udp.c  pubsub_udp.h  README.md  subscriber_udp.c

./udp_pcap_captures:
100k_EqA_vsEqB_normal_net_fast.pcap  15k_Each_net_fast_CDfirst.pcap      tcpdupm_udp_fast_normal_net_150k_full.pcap
10k_EqA_vsEqB_normal_net_fast.pcap   15k_Each_net_fast_v2.pcap           tcpdupm_udp_fast_normal_net_50k_each.pcap
12_andMine_Each_net_fast.pcap        25k_Each_net_fast.pcap              tcpdupm_udp_fast_normal_net_50k_each_v2.pcap
12_andMine_Each_slow_net_fast.pcap   25k_Each_net_fast_v2.pcap           tcpdupm_udp_fast_slow_net_150k_full.pcap
12_Each_net_fast.pcap                50k_EqA_vsEqB_normal_net_fast.pcap
```

Ilustración 2 Estructura del repositorio UDP

Se muestra el resultado del comando ls -R sobre el directorio del laboratorio, revelando tres subcarpetas: udp_asm con los archivos de assembly (broker_udp.s, publisher_udp.s, subscriber_udp.s y el Makefile), udp_C con los archivos en C (broker_udp.c, publisher_udp.c, subscriber_udp.c) y udp_pcap_captures con todas las capturas de tráfico generadas durante las pruebas.

Así mismo, se muestra la información relacionada con TCP:

```
maria@maria-VMware-Virtual-Platform:~/ISIS_2311_Lab3$ ls -la
total 192
drwxrwxr-x  3 maria   maria    4096 mar 31 19:54 .
drwxr-x--- 16 maria   maria    4096 mar 31 20:12 ..
-rwxrwxr-x  1 maria   maria   17160 mar 31 19:29 broker
-rwxrwxr-x  1 maria   maria   10096 mar 31 19:54 broker_tcp
-rw-rw-r--  1 maria   maria    8684 mar 31 19:54 broker_tcp.asm
-rw-rw-r--  1 maria   maria    9337 mar 31 13:37 broker_tcp.c
-rw-rw-r--  1 maria   maria    3216 mar 31 19:54 broker_tcp.o
drwxrwxr-x  9 maria   maria    4096 mar 31 19:54 .git
-rw-rw-r--  1 maria   maria     463 mar 30 23:15 .gitignore
-rwxrwxr-x  1 maria   maria   16560 mar 31 19:29 publisher
-rwxrwxr-x  1 maria   maria   10408 mar 31 19:39 publisher_tcp
-rw-rw-r--  1 maria   maria    7083 mar 31 19:39 publisher_tcp.asm
-rw-rw-r--  1 maria   maria    4201 mar 31 13:37 publisher_tcp.c
-rw-rw-r--  1 maria   maria    3232 mar 31 19:39 publisher_tcp.o
-rwxrwxr-x  1 maria   maria   16496 mar 31 19:29 subscriber
-rwxrwxr-x  1 maria   maria    9528 mar 31 19:34 subscriber_tcp
-rw-rw-r--  1 maria   maria    5054 mar 31 19:29 subscriber_tcp.asm
-rw-rw-r--  1 maria   maria    3417 mar 31 13:37 subscriber_tcp.c
-rw-rw-r--  1 maria   maria    2032 mar 31 19:34 subscriber_tcp.o
-rw-rw-r--  1 tcpdump tcpdump 14782 mar 31 20:10 tcp_pubsub.pcap
maria@maria-VMware-Virtual-Platform:~/ISIS_2311_Lab3$ S
```

Ilustración 3 Directorio del laboratorio con archivos TCP organizados

Se muestra el resultado del comando ls -la sobre el directorio ISIS_2311_Lab3/. Se pueden observar los seis archivos fuente TCP: tres en C (broker_tcp.c, publisher_tcp.c, subscriber_tcp.c) y tres en assembly x86-64 (broker_tcp.asm, publisher_tcp.asm, subscriber_tcp.asm), junto con sus respectivos ejecutables compilados (broker_tcp, publisher_tcp, subscriber_tcp) y archivos objeto (.o). También se incluye el archivo de captura de tráfico tcp_pubsub.pcap generado con tcpdump durante las pruebas.

5.2. Pruebas y comparacion

Ejecute simultáneamente al menos un broker, dos suscriptores y dos publicadores.

TCP

```
maria@maria-VMware-Virtual-Platform:~/ISIS_2311_Lab3$ nasm -f elf64 broker_tcp.asm -o broker_tcp.o && ld broker_tcp.o -o
 broker_tcp
maria@maria-VMware-Virtual-Platform:~/ISIS_2311_Lab3$ nasm -f elf64 publisher_tcp.asm -o publisher_tcp.o && ld publisher
_tcp.o -o publisher_tcp
maria@maria-VMware-Virtual-Platform:~/ISIS_2311_Lab3$ nasm -f elf64 subscriber_tcp.asm -o subscriber_tcp.o && ld subscri
ber_tcp.o -o subscriber_tcp
maria@maria-VMware-Virtual-Platform:~/ISIS_2311_Lab3$
```

Ilustración 4 Compilación de los archivos TCP assembly

Se muestran los comandos de compilación de los tres archivos assembly x86-64. Primero se usa nasm -f elf64 para convertir cada archivo .asm en un archivo objeto .o, y luego se genera el ejecutable final con ld. La ausencia de errores confirma que los archivos están correctamente escritos y listos para ejecutarse.

```
[Broker] Mensaje reenviado a subscribers.
[Broker] Mensaje reenviado a subscribers.
[Broker] Mensaje reenviado a subscribers.
[Broker] Mensaje reenviado a subscribers.
[Broker] Mensaje reenviado a subscribers.
[Broker] Mensaje reenviado a subscribers.
[Broker] Mensaje reenviado a subscribers.
[Broker] Mensaje reenviado a subscribers.
[Broker] Publisher desconectado.
[Broker] Publisher conectado. Recibiendo mensajes...
[Broker] Mensaje reenviado a subscribers.
[Broker] Mensaje reenviado a subscribers.
[Broker] Mensaje reenviado a subscribers.
[Broker] Publisher desconectado.
```

```
[Subscriber] >> [Colombia vs Brasil] Inicio del partido
[Colombia vs Brasil] Tiro de esquina para el equipo local
[Colombia vs Brasil] GOOOL del equipo local al minuto 12
[Colombia vs Brasil] Tarjeta amarilla al numero 8 del visit
ante
[Colombia vs Brasil] Fin del primer tiempo. Marcador: 1-0
[Colombia vs Brasil] Inicio del segundo tiempo
[Colombia vs Brasil] Cambio: jugador 11 entra por jugador 7
[Colombia vs Brasil] GOOOL del visitante al minuto 67. Empa
te 1-1
[Subscriber] >> [Colombia vs Brasil] Tarjeta roja al numero
 4 del local al minuto 88
[Subscriber] >> [Colombia vs Brasil] Fin del partido. Resul
tado final: 1-1
```

```
[Subscriber] >> [Colombia vs Brasil] Inicio del partido
[Colombia vs Brasil] Tiro de esquina para el equipo local
[Colombia vs Brasil] GOOOL del equipo local al minuto 12
[Colombia vs Brasil] Tarjeta amarilla al numero 8 del visita
nte
[Colombia vs Brasil] Fin del primer tiempo. Marcador: 1-0
[Colombia vs Brasil] Inicio del segundo tiempo
[Colombia vs Brasil] Cambio: jugador 11 entra por jugador 7
[Colombia vs Brasil] GOOOL del visitante al minuto 67. Empat
e 1-1
[Subscriber] >> [Colombia vs Brasil] Tarjeta roja al numero
4 del local al minuto 88
[Subscriber] >> [Colombia vs Brasil] Fin del partido. Result
ado final: 1-1
```

```
[Colombia vs Brasil] GOOOL del visitante al minuto 67. Empa
te 1-1
[Colombia vs Brasil] Tarjeta roja al numero 4 del local al
minuto 88
[Colombia vs Brasil] Fin del partido. Resultado final: 1-1
[Publisher] Transmision finalizada.
maria@maria-VMware-Virtual-Platform:~/ISIS_2311_Lab3$
```

```
te 1-1
[Colombia vs Brasil] Tarjeta roja al numero 4 del local al
minuto 88
[Colombia vs Brasil] Fin del partido. Resultado final: 1-1
[Publisher] Transmision finalizada.
maria@maria-VMware-Virtual-Platform:~/ISIS_2311_Lab3$ S
```

Ilustración 5 Ejecución simultanea TCP: broker + 2 subscribers + 2 publishers

Se muestra la ejecución simultánea del sistema TCP implementado en assembly usando tmux con cuatro paneles. En el panel superior izquierdo el broker confirma la recepción y reenvío de cada mensaje, así como la conexión y desconexión de los publishers. En los paneles superior derecho e inferior izquierdo los dos subscribers reciben en tiempo real los 10 eventos del partido "Colombia vs Brasil". En el panel inferior derecho se observa que ambos publishers finalizaron su transmisión exitosamente. Todos los mensajes llegaron completos y en orden a los dos subscribers, confirmando el correcto funcionamiento del sistema TCP en assembly x86-64.

UDP

```
server1@server1:~/labs/lab3/ISIS_2311_Lab3/udp_asm$ make clean && make
rm -f broker_udp_asm publisher_udp_asm subscriber_udp_asm
gcc -O1 -std=c11 -D_DEFAULT_SOURCE -no-pie -o broker_udp_asm broker_udp.s
gcc -O1 -std=c11 -D_DEFAULT_SOURCE -no-pie -o publisher_udp_asm publisher_udp.s
gcc -O1 -std=c11 -D_DEFAULT_SOURCE -no-pie -o subscriber_udp_asm subscriber_udp.s
```

Ilustración 6 Compilación de los archivos UDP en assembly

Se muestra el resultado del comando make clean && make sobre el directorio udp_asm, que compila los tres archivos assembly UDP usando gcc con las flags -O1 -std=c11 -D_DEFAULT_SOURCE -no-pie, generando los ejecutables broker_udp_asm, publisher_udp_asm y subscriber_udp_asm.

```
server1@server1:~/labs/lab3/ISIS_2311_Lab3/udp_asm$ ./broker_udp_asm 9000
[broker] Escuchando UDP en el puerto 9000. Ctrl+C para salir.
```

Ilustración 7 Broker UDP en ejecución

Se muestra el broker UDP escuchando en el puerto 9000, listo para recibir conexiones de publishers y subscribers.

```
server2@server2:~/labs/lab3/ISIS_2311_Lab3$ ./subscriber_udp 192.168.1.18 9000 EquipoA_vs_EquipoB
[subscriber] Registrado SUB 'EquipoA_vs_EquipoB' (23 bytes enviados al broker)
[subscriber] Suscripciones enviadas. Esperando noticias (Ctrl+C)...

[subscriber] [192.168.1.18:9000] (datagrama no estándar) OK SUB EquipoA_vs_EquipoB
```

Ilustración 8 Subscriber 1 UDP suscrito al topic EquipoA_vs_EquipoB

Se muestra el subscriber 1 registrado exitosamente en el broker y esperando actualizaciones del partido EquipoA_vs_EquipoB.

```
server2@server2:~/labs/lab3/ISIS_2311_Lab3$ ./subscriber_udp 192.168.1.18 9000 EquipoC_vsEquipoD
[subscriber] Registrado SUB 'EquipoC_vsEquipoD' (22 bytes enviados al broker)
[subscriber] Suscripciones enviadas. Esperando noticias (Ctrl+C)...

[subscriber] [192.168.1.18:9000] (datagrama no estándar) OK SUB EquipoC_vsEquipoD
```

Ilustración 9 Subscriber 2 UDP suscrito al topic EquipoC_vs_EquipoD

Se muestra el subscriber 2 registrado exitosamente en el broker y esperando actualizaciones del partido EquipoC_vs_EquipoD.

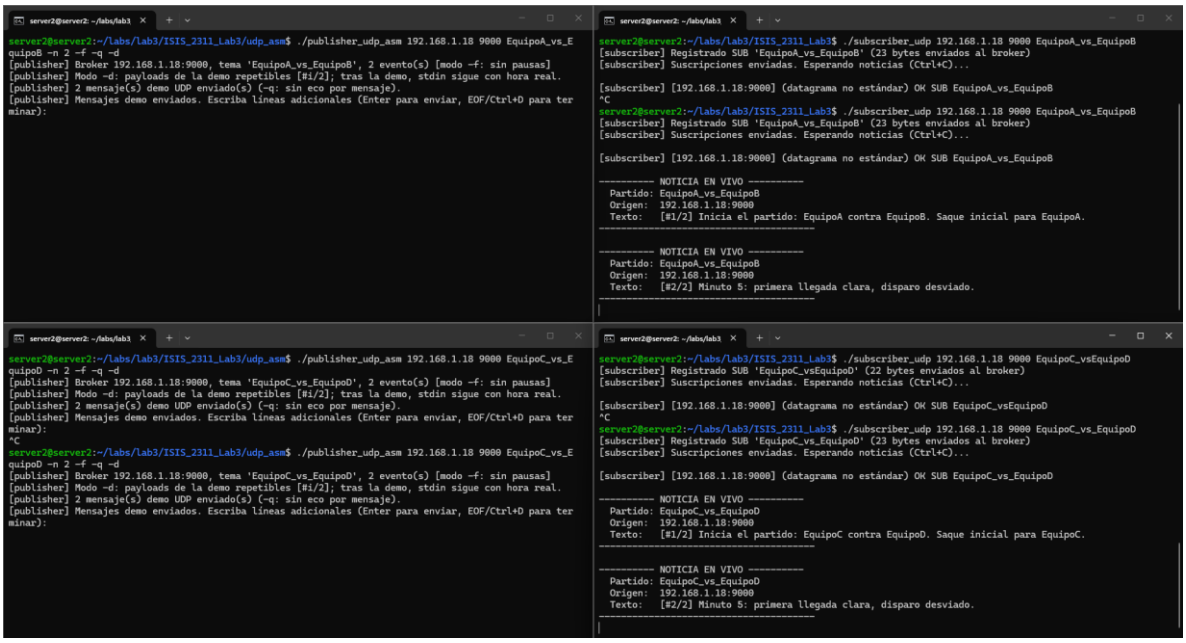

Ilustración 10 Ejecución simultánea UDP: broker + 2 subscribers + 2 publishers

Se muestra la ejecución simultánea del sistema UDP con cuatro terminales. Los dos publishers envían 2 mensajes cada uno en modo determinístico para facilitar la visualización. Los dos subscribers reciben correctamente las actualizaciones de sus respectivos partidos.

Captura de tráfico con Wireshark

Nota; por facilidad se uso tcpdump dentro de la MV asociada al bróker en Linux, para capturar de manera correcta el trafico de la aplicación.

TCP

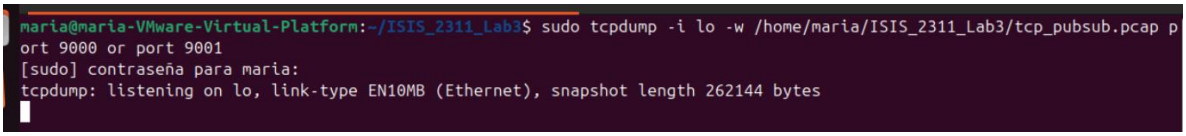

Ilustración 11 Comando tcpdump para captura de tráfico TCP

Se muestra el comando utilizado para capturar el tráfico TCP: sudo tcpdump -i lo -w tcp_pubsub.pcap port 9000 or port 9001. Se especifica la interfaz loopback (lo) ya que todos los procesos corren en la misma máquina, y se filtra únicamente el tráfico en los puertos 9000 y 9001 correspondientes al broker.

```
maria@maria-VMware-Virtual-Platform:~/ISIS_2311_Lab3$ sudo tcpdump -i lo -w /home/mari
ort 9000 or port 9001
[sudo] contraseña para maria:
tcpdump: listening on lo, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C130 packets captured
260 packets received by filter
0 packets dropped by kernel
maria@maria-VMware-Virtual-Platform:~/ISIS_2311_Lab3$
```

Ilustración 12 Resultado de la captura TCP con tcpdump

Se muestra el resultado al finalizar la captura. Se capturaron 130 paquetes en total y 0 paquetes fueron descartados por el kernel, confirmando que TCP no presentó ninguna pérdida de paquetes durante la transmisión. Esto contrasta directamente con el comportamiento de UDP, donde sí se evidenciaron pérdidas al aumentar el volumen de mensajes como se mostrará más adelante.

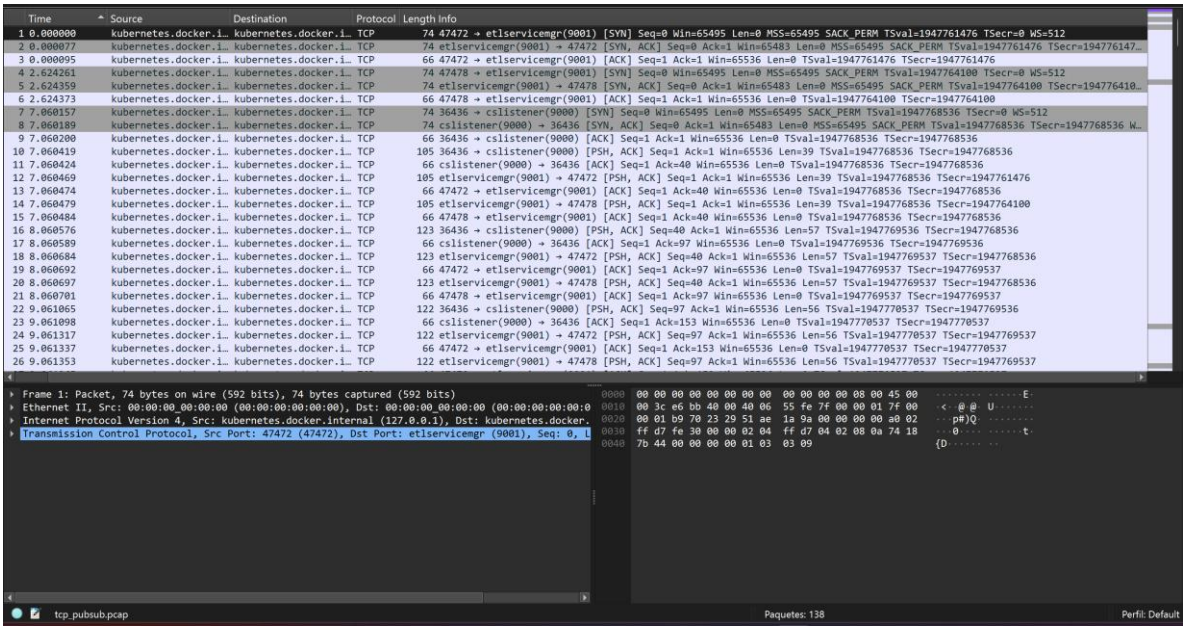

Ilustración 13 Archivo tcp_pubsub.pcap abierto en Wireshark

Se muestra la vista general del archivo de captura TCP en Wireshark. Se observan los handshakes de tres vías (SYN, SYN-ACK, ACK) al inicio de cada conexión, seguidos de los mensajes de datos con flags PSH, ACK. Por cada mensaje enviado por un publisher el broker genera dos reenvíos hacia los subscribers, reflejado en los paquetes duplicados hacia el puerto 9001.

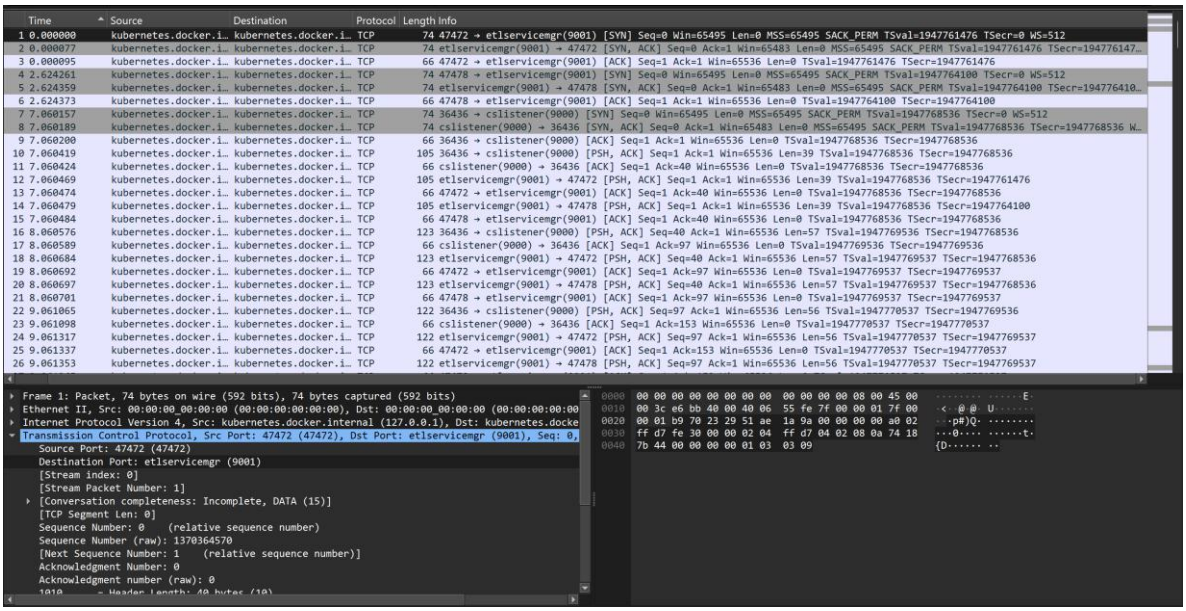

Ilustración 14 Detalle del primer paquete TCP capturado

Se muestra el detalle del primer paquete de la captura al hacer click sobre él en Wireshark. Se puede observar que es un paquete de 74 bytes, con dirección origen y destino 127.0.0.1 (loopback), puerto origen 47472 y puerto destino 9001 correspondiente al servidor de subscribers. El flag SYN indica que es el inicio del handshake TCP. En los campos de control se destacan el número de secuencia relativo (Seq=0), el número de secuencia raw (1947364570), el tamaño de ventana (Win=65495) y las opciones MSS=65495, SACK y timestamps, que son parte del mecanismo de confiabilidad y control de flujo de TCP.

UDP

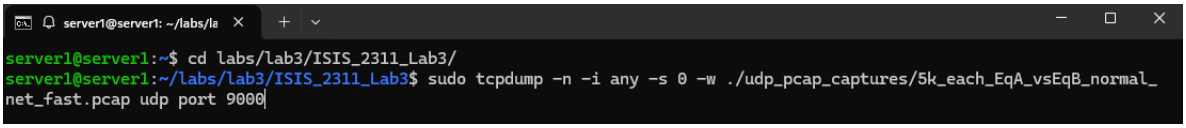

Ilustración 15 Comando tcpdump para captura de tráfico UDP

Comando usado para capturar el tráfico UDP con tcpdump, filtrando por el puerto 9000 del broker. Se enviaron 5000 paquetes desde cada publisher para evidenciar el comportamiento de UDP bajo alta carga.

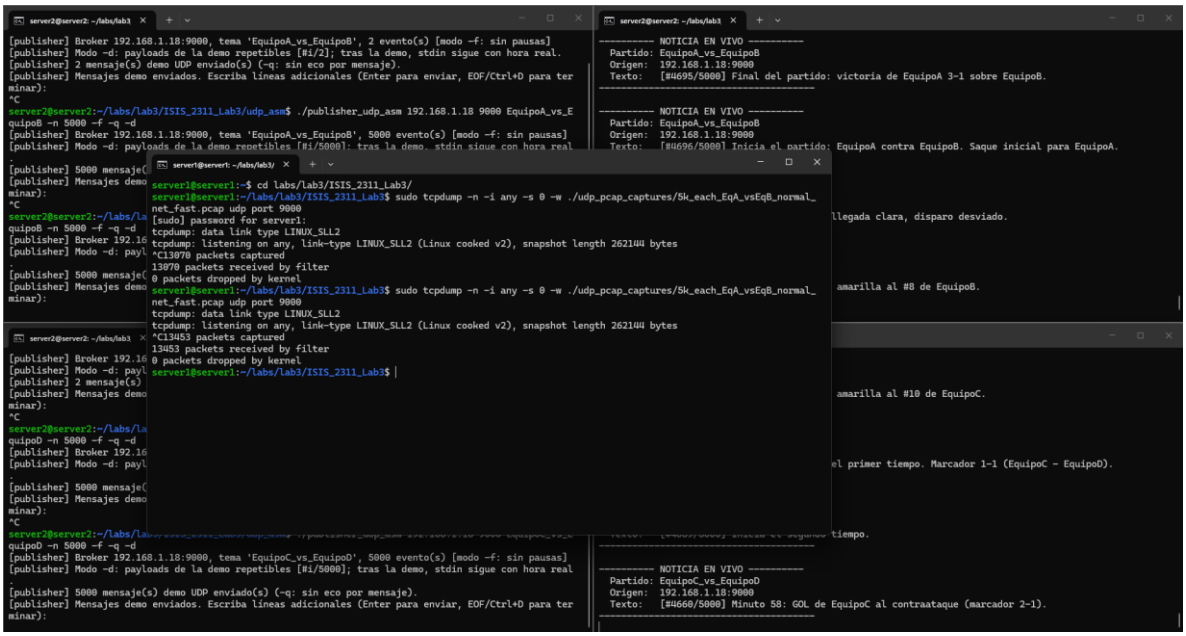

Ilustración 16 Resultado de la captura UDP con pérdida de paquetes

Al cerrar tcpdump tras enviar 5000 paquetes determinísticos por publisher, se observa que el subscriber 2 recibió el paquete #4660/5000 como último en lugar del esperado #5000/5000, evidenciando pérdida de paquetes en UDP bajo alta carga.

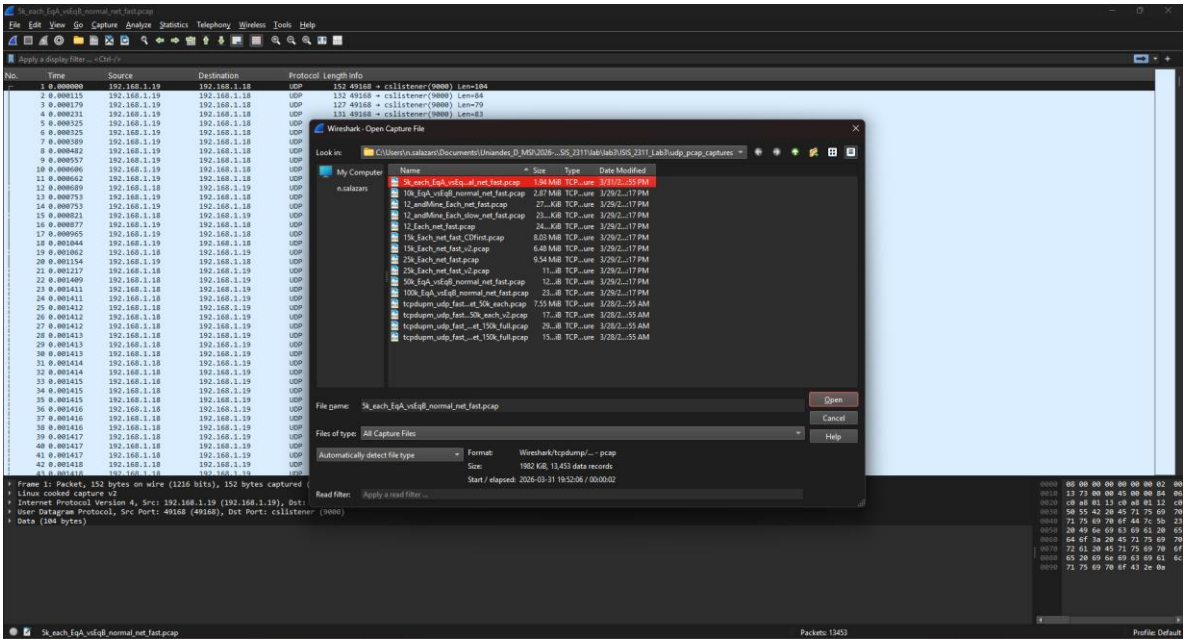

Ilustración 17 Archivo udp_pubsub.pcap abierto en Wireshark

Vista general de la captura UDP en Wireshark. A diferencia de TCP, no se observan handshakes ni ACKs. Cada fila es un datagrama independiente enviado directamente, mostrando el menor overhead de UDP.

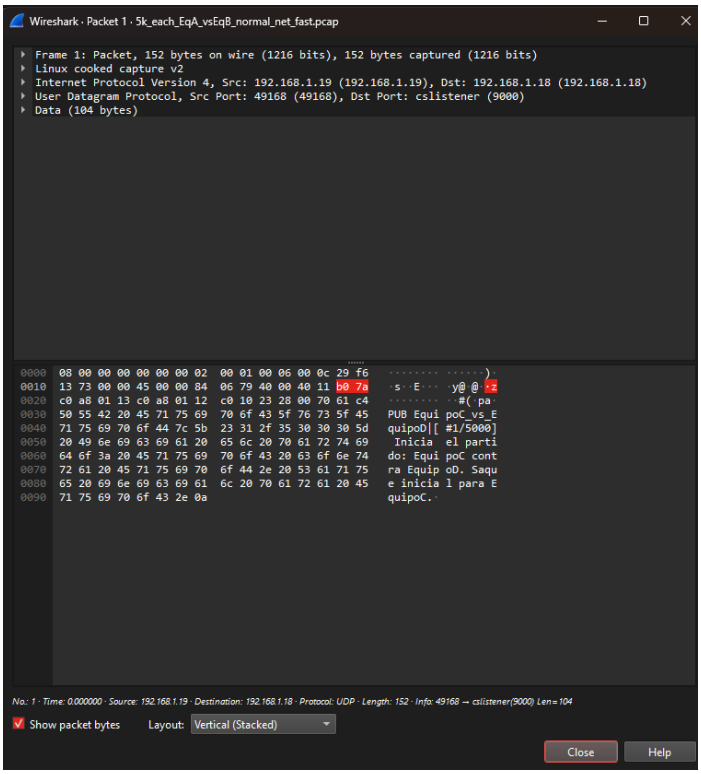

Ilustración 18 Detalle del primer paquete UDP capturado

Detalle del primer datagrama UDP. Se observa el protocolo UDP, el puerto origen del publisher y el puerto destino 9000 del broker. No hay campos de secuencia ni control de flujo, reflejando la simplicidad del protocolo.

Analice las siguientes preguntas:

a. En TCP:

   i. ¿los mensajes llegan <u>completos</u> y en orden?

Sí, todos los mensajes llegaron completos y en orden a los dos subscribers. Durante las pruebas, cada evento del partido fue recibido en la misma secuencia en que fue enviado por el publisher. Como se observa en la Ilustración 13, los números de secuencia (Seq) aumentan ordenadamente y cada mensaje con flag PSH, ACK es confirmado por un ACK del receptor. A diferencia de UDP, en ningún momento se presentó pérdida o desorden en los mensajes.

   ii. ¿Cómo maneja TCP la confiabilidad y el control de flujo?

TCP garantiza la entrega mediante confirmaciones (ACK), o sea, por cada mensaje enviado, el receptor responde con un ACK. Si el emisor no recibe esa confirmación en

un tiempo determinado, retransmite el paquete automáticamente sin que la aplicación tenga que hacer nada. El control de flujo se maneja con el campo Window Size (Win), visible en la Ilustración 14 con valor Win=65536, que indica cuántos bytes puede recibir el broker antes de necesitar confirmación. Esto evita que el emisor sature al receptor con más datos de los que puede procesar.

b. En UDP:

i. ¿Qué evidencias hay de perdida o desorden en los mensajes?

Para esto, fue necesario hacer un feature como flag -d 'deterministico', en el cual las noticias tuvieran un 'label' que las representaba en el orden que el pub las iba creando, es decir, si el pub mandaba 10 mensajes, el primero iba como [1/10], el segundo como [2/10] y así, hasta el [10/10]. Pudimos evidenciar que con un numero bajo de paquetes, como 100, los paquetes llegaban en orden y completos (no se presentaban perdidas), no obstante, conforme aumenta el numero, si que se presentan perdidas, como podemos evidenciar en la siguiente imagen:

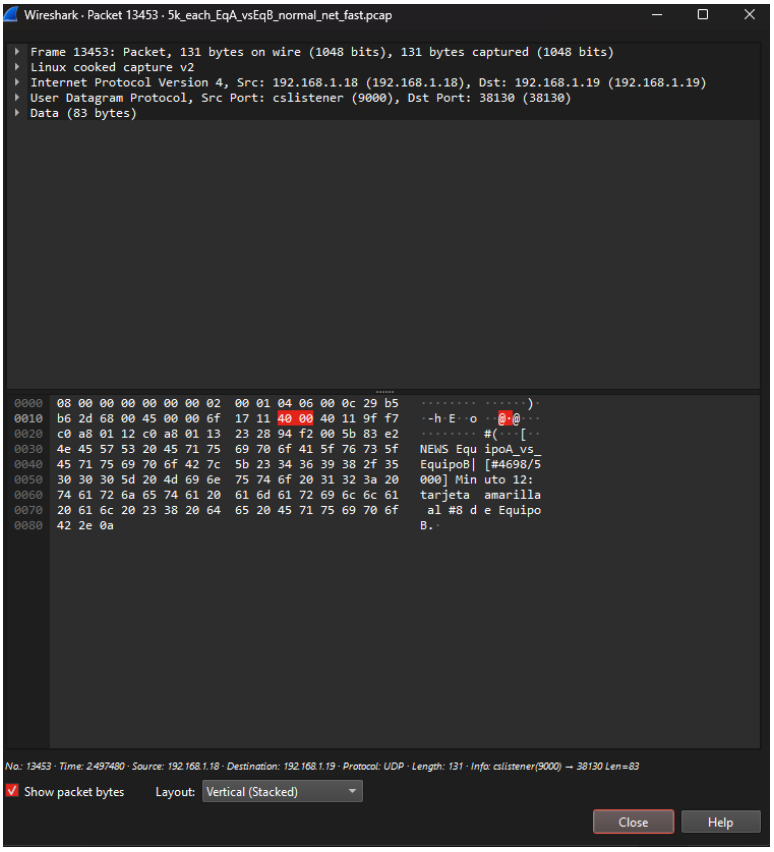

Ilustración 19 Último paquete UDP recibido fuera de orden

Se muestra el último paquete capturado por tcpdump. El paquete recibido no corresponde al #5000 esperado, siendo este el enviado por el publisher con los flags -

n 5000 y -d (modo determinístico). Esto confirma que UDP presentó pérdida de paquetes al manejar un volumen alto de mensajes.

c. ¿Qué diferencias observa en el manejo de la conexión entre ambos protocolos?

La diferencia principal está en cómo se establece la comunicación. TCP negocia una conexión antes de enviar cualquier dato mediante el handshake de tres vías (SYN, SYN-ACK, ACK), claramente visible en los primeros paquetes de la Ilustración 13. Esta conexión se mantiene activa durante toda la sesión y se cierra limpiamente con paquetes FIN. UDP en cambio no establece ninguna conexión: cada datagrama se envía directamente sin negociación previa. En términos prácticos, con TCP el broker sabe exactamente cuándo un cliente se conecta o se desconecta, mientras que con UDP simplemente deja de recibir paquetes sin ninguna notificación.

6.0. Trabajo propuesto

1. Implementar el sistema publicador-suscriptor en TCP y en UDP.

Este sistema se encuentra en el repositorio entregado: https://github.com/panisema2003/ISIS_2311_Lab3.git

2. No está permitido el uso de librerías de implementación de sockets a menos que se especifique punto a punto la interacción del programa con cada función de la librería, debe estar totalmente documentado.

Nota: Para la implementación TCP, la documentación de cada función de librería utilizada se encuentra directamente en los archivos fuente .c y .asm, con comentarios explicando qué hace cada función, sus parámetros y su valor de retorno. Para la implementación UDP, la documentación se encuentra en el README.md.

3. Verificación de entrega de mensajes

TCP

Como se observa en la Ilustración 5, los 10 eventos del partido fueron enviados correctamente desde cada publisher y recibidos en su totalidad por los dos subscribers, sin pérdida ni desorden. Esto confirma el correcto funcionamiento del sistema pub-sub TCP tanto en la implementación en C como en assembly.

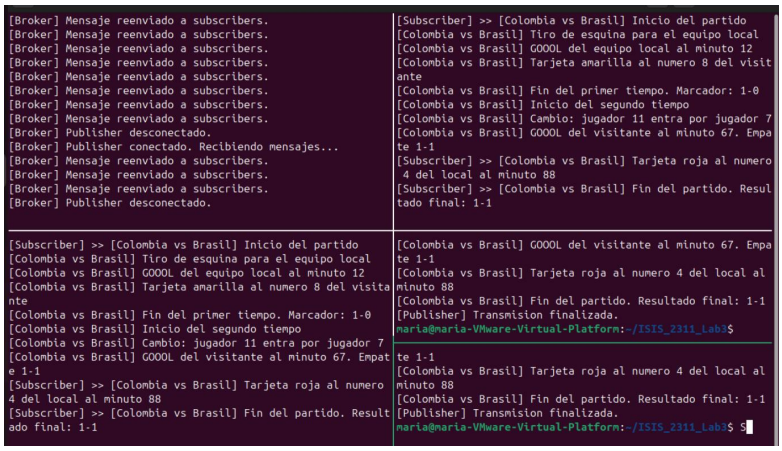

Ilustración 5 Ejecución simultanea TCP: broker + 2 subscribers + 2 publishers

Se muestra la ejecución del sistema TCP en assembly usando tmux. En el panel superior izquierdo el broker confirma la recepción y reenvío de cada mensaje. En los demás paneles los dos subscribers reciben los 10 eventos del partido "Colombia vs Brasil" completos y en orden, y ambos publishers finalizan su transmisión exitosamente.

UDP

Con UDP y solo 10 paquetes los mensajes llegaron sin pérdida alguna, demostrado por el modo determinístico como se ve en la siguiente imagen.

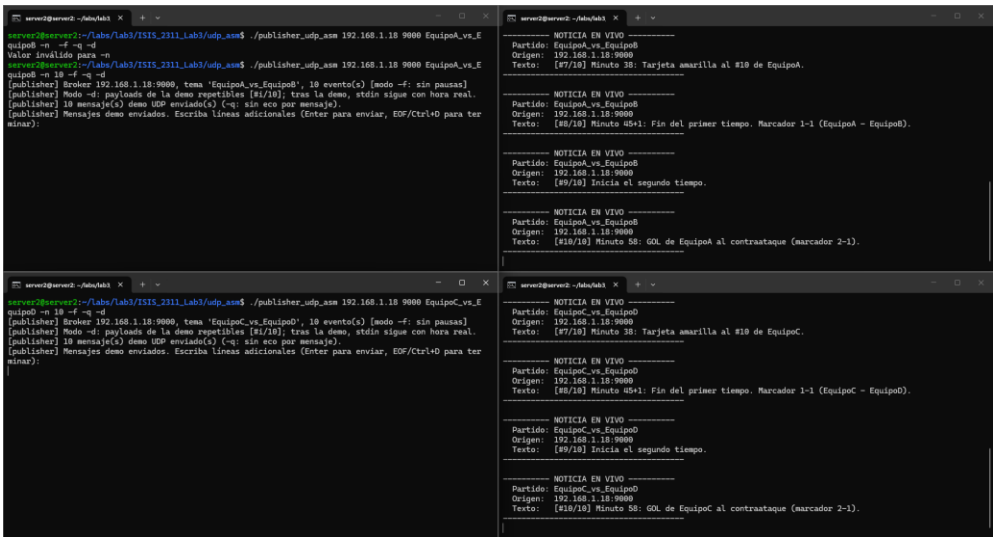

Ilustración 20 Verificación de 10 mensajes UDP sin pérdida

Se muestra la ejecución simultánea del sistema UDP con cuatro terminales. Los dos publishers (izquierda) envían 10 eventos cada uno a sus respectivos partidos: EquipoA_vs_EquipoB y EquipoC_vs_EquipoD. Los dos subscribers (derecha) reciben correctamente todos los mensajes en tiempo real, incluyendo tarjetas amarillas,

marcadores y el inicio del segundo tiempo, confirmando que con un volumen bajo de 10 mensajes UDP no presenta pérdidas.

4. Captura de tráfico y archivos .pcap
    a. tcp_pubsub.pcap
    b. udp_pubsub.pcap

TCP

Se capturó el tráfico TCP usando tcpdump en la interfaz loopback, generando el archivo tcp_pubsub.pcap. Este archivo se puede encontrar en el repositorio del laboratorio. Como se evidencia en la Ilustración 12, no se presentó ninguna pérdida de paquetes durante la captura.

UDP

Se generaron múltiples archivos de captura bajo distintas condiciones de red y volumen de mensajes. Todos los archivos .pcap se pueden encontrar dentro de la carpeta udp/udp_pcap_captures/ del repositorio.

5. Tabla comparativa TCP vs UDP

Criterio / Protocolo	**UDP**	**TCP**
Confiabilidad	No garantiza la entrega. Los mensajes pueden perderse sin notificación.	Garantiza la entrega mediante ACKs y retransmisión automática.
Orden de entrega	No garantiza orden. Los paquetes pueden llegar desordenados.	Garantiza que los mensajes llegan en el mismo orden en que fueron enviados.
Perdida de mensajes	Puede tener, especialmente con gran número de paquetes o una red lenta o con mucho trafico	No tiene, y si tiene, corrige.
Overhead de cabeceras/protocolo	Cabecera fija, ligera y de 8 bytes. Sin campos de control adicionales.	Cabecera mas grande que udp, con un mínimo de 20 bytes, con campos adicionales como números de secuencia, ACK, Window Size y timestamps.

Tabla 1 Comparación

6. Preguntas de análisis
    a. ¿Qué ocurría si en lugar de dos publicadores (partidos transmitidos) hubiera cien partidos simultáneos? ¿Como impactaría esto en el desempeño del broker bajo TCP y bajo UDP?

UDP

Con 100 publishers, el broker UDP tendría que procesar muchas más operaciones de recvfrom, parsing y sendto por segundo. Al no mantener estado por conexión escala mejor que TCP, pero el riesgo de pérdida de paquetes aumenta significativamente por la saturación de la cola del socket o de la red.

TCP

Con 100 publishers simultáneos, el broker debería mantener 100 conexiones TCP abiertas al mismo tiempo, lo que aumenta considerablemente el consumo de memoria y CPU. Además, por cada mensaje recibido el broker debe gestionarlo y reenviarlo a todos los subscribers, multiplicando la carga. A diferencia de UDP, TCP también debe manejar ACKs, retransmisiones y control de flujo por cada conexión, lo que hace que escale peor bajo alta concurrencia.

    b. Si un gol se envía como mensaje desde el publicador y un suscriptor no lo recibe en UDP, ¿qué implicaciones tendría para la aplicación real? ¿Por qué TCP maneja mejor este escenario?

El usuario puede quedarse con un valor incorrecto del partido en vivo, teniendo inconsistencias, lo cual podria ser terrible, por ejemplo, en la copa del mundo de futbol que ya casi viene, podria pensar que su equipo paso, cuando en realidad perdió. TCP maneja mucho mejor este escenario, pues garantiza integridad de los datos enviados, es decir, el marcador siempre será el que corresponde al lado del pub.

    c. En un escenario de seguimiento en vivo de partidos, ¿qué protocolo (TCP o UDP) resultaría más adecuado? Justifique con base en los resultados de la práctica

Consideramos que depende de la prioridad de la aplicación. Si lo más importante es la velocidad y que los eventos lleguen en tiempo real, UDP sería más adecuado ya que tiene menor latencia y no espera confirmaciones. Sin embargo, si la prioridad es la integridad del marcador, TCP es la mejor opción ya que garantiza que todos los mensajes llegan completos y en orden. En las pruebas, TCP no presentó ninguna pérdida mientras que UDP sí tuvo pérdidas al aumentar el volumen. Para una aplicación de noticias deportivas en vivo, lo ideal sería una combinación: UDP para

eventos en tiempo real donde un pequeño retraso es inaceptable, y TCP para mensajes críticos como el marcador final.

d. Compare el overhead observado en las capturas Wireshark entre TCP y UDP. ¿Cuál protocolo introduce más cabeceras por mensaje? ¿Cómo influye esto en la eficiencia?

El overhead es claramente mayor en TCP, como se muestra en la tabla del punto 6.5. La cabecera TCP tiene un mínimo de 20 bytes con campos adicionales como números de secuencia, ACK y Window Size, mientras que UDP solo tiene 8 bytes fijos. Esto significa que TCP envía más bytes por cada mensaje, lo que reduce la eficiencia en términos de ancho de banda utilizado.

e. Si el marcador de un partido llega desordenado en UDP (por ejemplo, primero se recibe el 2–1 y luego el 1–1), ¿qué efectos tendría en la experiencia del usuario? ¿Cómo podría solucionarse este problema a nivel de aplicación?

Sin duda la experiencia de usuario seria una reacción negativa, pues no tendría ningún sentido lógico recibir en ese orden las noticias. A nivel de aplicación, podria solucionarse por medio de la documentación (numeración) de las noticias enviadas, similar al modo -d de UDP implementado en el lab, en el que se puede ver a que noticia corresponde cada mensaje, en este caso 2-1 podria ser noticia 3, por lo que si el usuario no ha recibido antes 2, pues sabe que algo paso antes y no le llegó.

f. ¿Cómo cambia el desempeño del sistema cuando aumenta el número de suscriptores interesados en un mismo partido? ¿Qué diferencias se observaron entre TCP y UDP en este aspecto?

TCP

Con más subscribers, el broker debe mantener más conexiones activas y hacer más operaciones de send() por cada mensaje recibido. Con dos subscribers funcionó sin problema, pero con muchos más el consumo de memoria y CPU sería considerable.

UDP

UDP escala mejor en este aspecto porque no mantiene estado por cliente. El broker simplemente envía datagramas sin gestionar conexiones individuales, aunque a costa de no garantizar la entrega.

g. ¿Qué sucede si el broker se detiene inesperadamente? ¿Qué diferencias hay entre TCP y UDP en la capacidad de recuperación de la sesión?}

UDP

No hay cierre de conexión que notifique la pérdida, ya que no existe sesión. Los publishers y subscribers podrían seguir enviando o esperando indefinidamente hasta recibir un error de destino inalcanzable o un timeout, sin ninguna notificación clara.

TCP

Cuando el broker cae, los subscribers detectan la desconexión inmediatamente porque recv() retorna 0. Esto permite manejar el error en la aplicación y notificar al usuario. Sin embargo, TCP no reconecta automáticamente, por lo que el subscriber debería iniciar una nueva conexión manualmente cuando el broker vuelva.

h. ¿Cómo garantizar que todos los suscriptores reciban en el mismo instante las actualizaciones críticas (por ejemplo, un gol)? ¿Qué protocolo facilita mejor esta sincronización y por qué?

Si bien consideramos que es imposible "garantizar" que todos los subs reciban al mismo instante actualizaciones críticas, creemos que con UDP multicast se priorizaría la velocidad, ya que un solo paquete llegaría a todos los suscriptores simultáneamente. Con TCP, el broker envía secuencialmente a cada subscriber, lo que introduce una pequeña diferencia de tiempo entre el primero y el último. En la práctica con dos subscribers esta diferencia es imperceptible, pero escalaría con mayor número de clientes.

i. Analice el uso de CPU y memoria en el broker cuando maneja múltiples conexiones TCP frente al manejo de datagramas UDP. ¿Qué diferencias encontró?

TCP

El broker TCP consume más recursos porque mantiene estado por cada conexión activa: buffers, números de secuencia y timers de retransmisión. En las pruebas con dos publishers y dos subscribers el impacto fue mínimo y el sistema funcionó con fluidez, pero con muchas conexiones simultáneas la diferencia sería notable.

UDP

El broker UDP es más liviano porque simplemente recibe datagramas y los reenvía sin mantener estado por cliente, lo que lo hace más eficiente en escenarios de alta concurrencia.

j. Si tuviera que diseñar un sistema real de transmisión de actualizaciones de partidos de fútbol para millones de usuarios, ¿elegiría TCP, UDP o una

combinación de ambos? Justifique con base en lo observado en el laboratorio.

Por ahora, creemos que lo mejor sería una combinación de ambos, pues si bien con UDP la velocidad es priorizada, seria ideal contar con una garantía de la integridad de los mensajes como contamos con TCP. Quizas la solución sea implementar QUIC, pero nos daremos cuenta la próxima semana cuando entreguemos ese protocolo de transporte :) .

Bibliografía

Quiroga, N. (2025). *Laboratorio #3: Análisis capa de transporte y sockets* (Versión 1). Departamento de Ingeniería de Sistemas y Computación, Universidad de los Andes. Infraestructura de Comunicaciones, ISIS 2311.